



《系统软件原理与设计III-数据库系统》

第8章 数据库故障恢复

计算机科学与工程学院（网络空间安全学院）

任课教师：胡旺

2025年12月16日星期二



本章内容



1

事务概念

2

数据库恢复概述

3

数据库恢复实现技术

4

故障的种类及恢复策略

5

*RAID



■ 产生不一致的结果

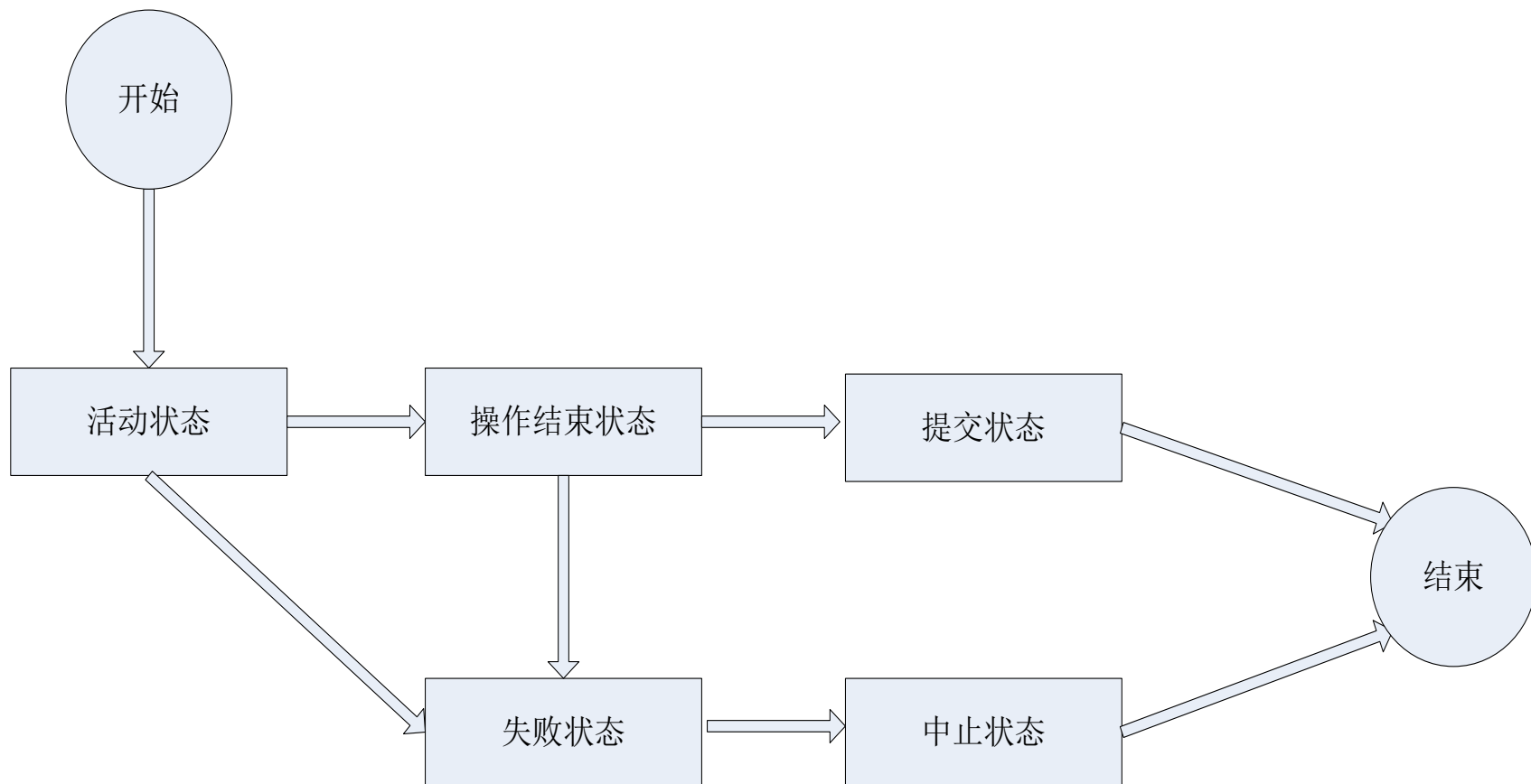
- ◆调价：故障可能使部分价格调整而另一部分价格没有调整。

■ 并发执行的错误

- ◆并行执行应用程序可以提高性能
- ◆但可能数据相互覆盖
- ◆例如：程序1将A提价10%，程序2将A涨价10元。

■ 何时更新数据库

- ◆读数据库：将数据库中的数据先从磁盘读入内存，然后再将值赋予一个变量。
- ◆写数据库：先将变量的值写入内存，然后再由内存写入磁盘。
- ◆减少磁盘I/O操作与防止数据不一致的权衡（内存中数据的存放时间）





■ 事务的显式定义

BEGIN TRANSACTION

SQL 语句1

SQL 语句2

...

COMMIT

END TRANSACTION

BEGIN TRANSACTION

SQL 语句1

SQL 语句2

...

ROLLBACK

END TRANSACTION

■ 没有显式地定义事务时，DBMS按缺省为自动划分事务



■ 原子性 (Atomicity)

- ◆ 事务的所有操作在数据库中要么全部正确反映，要么全部不反映。
- ◆ 解决不一致问题。
- ◆ 在系统崩溃后，DBMS将恢复或撤销系统崩溃时处于活动状态的事务对数据库产生的影响，从而保证事务的原子性。
- ◆ 事务原子性由事务管理部件 (Transaction-Management Component) 处理。

■ 一致性 (Consistency)

- ◆ 是指当事务完成时，必须使所有数据都具有一致的状态。
- ◆ 主要由应用开发人员来确保。



■ 隔离性 (Isolation)

- ◆ 当多个事务并发执行时，一个事务的执行不能被其他事务干扰。
- ◆ 解决前面提到的并发执行带来的错误问题。
- ◆ 由于性能的原因，需要对事务进行交叉调度，但交错调度的效果应该和某个串行调度的结果是一致的。
- ◆ 隔离性通过数据库系统中的并发控制部件 (Concurrency-Control Component) 处理。

■ 持久性 (Durability)

- ◆ 一个事务一旦提交，它对数据库中数据的改变应该是永久性的，即使系统可能出现故障。
- ◆ 持久性由数据库系统中的恢复管理部件 (Recovery-Management Component) 的软件部件负责。



本章内容



1

事务概念



2

数据库恢复概述

3

数据库恢复实现技术

4

故障的种类及恢复策略

5

*RAID



- 数据库系统对付故障的两种措施
 - ◆ 预防故障：尽可能提高系统的可靠性；
 - ◆ 在系统发生故障后，把数据库恢复到一致状态。
- 恢复机制涉及两个关键问题
 - ◆ 如何建立冗余数据
 - ◆ 如何利用冗余数据实施数据库恢复
- 恢复技术是衡量数据库管理系统优劣的重要指标



本章内容



1

事务概念

2

数据库恢复概述



3

数据库恢复实现技术

4

故障的种类及恢复策略

5

*RAID



■ 恢复机制常用的技术

- ◆ 记录日志文件
- ◆ 数据转储



- 日志是DBMS用来记录事务对数据库的更新操作的文件，是日志记录的序列
- 日志记录描述内容主要包括：
 - ◆ 事务标识符：执行写操作事务的唯一标识符。
 - ◆ 数据项标识符：事务操作对象的唯一标识符。
 - ◆ 前像（BI）：更新前数据的旧值。
 - ◆ 后像（AI）：更新后数据的新值。
- 日志记录形式包括：
 - ◆ <T START>：表示事务T已开始。
 - ◆ <T COMMIT>：表示事务T已提交。
 - ◆ <T ABORT>：表示事务T不能成功完成，已中止。
 - ◆ <T, X, V1, V2>：表示事务T对数据项X执行写操作。写之前的旧值为V1，写之后的新值为V2。



■ 日志要求

- ◆ 每次事务执行写操作，必须在数据库修改前建立此次写操作的日志记录
- ◆ 日志必须存储在稳定存储器上
- ◆ 稳定存储器中的日志记录顺序必须与写入日志缓冲区中的日志记录顺序完全一样（块写）

■ 先写日志规则

- ◆ 在主存中的数据块输出到数据库前，所有与该数据块中数据有关的日志记录必须已输出到稳定存储器上



■ 事务的执行时，后像（AI）在事务提交后才写入数据库

- ◆ 日志中记录所有的数据库修改
- ◆ 一个事务的所有写操作延迟到事务的操作结束时才执行，以保证事务的原子性。

事务T的执行步骤

- 在T开始执行前，向日志中写入记录<T START>
- T的一次write (X) 操作导致向日志中写入一条新记录
- 最后，当T全部操作结束，向日志中写入记录<T COMMIT>

■ 事务恢复

- ◆ 忽略未完成的事务；重复已提交事务的影响
- ◆ 恢复过程Redo (Ti)：将事务 T i更新的所有数据项的值设为新值

■ 简化日志内容结构

- ◆ 日志记录<T, X, V1>：事务 T 对数据项 X 执行写操作，写入新值V1



■ 举例：后像后写

◆ 药品价格调整：设T1事务中，药品A上调10%，药品B下浮5%。

T1:

```
read(A)
A:=A+A*0.1
write(A)
read(B)
B:=B-B*0.05
Write(B)
```

A=20, B=30

步骤	动作	日志
1		<T1 START>
2	READ(A)	
3	A:=A+A*0.1	
4	Write (A)	<T1,A,22>
5	READ(B)	
6	B:=B-B*0.05	
7	Write(B)	<T1,B,28.5>
8		<T1,COMMIT>
9	写A到磁盘	
11	写B到磁盘	



■ 恢复管理器执行步骤：

- ◆① 从后向前扫描日志，将提交的事务放入队列redo-list。
- ◆② 从前往后扫描日志。对遇到的每一 $\langle T, X, V1 \rangle$ 记录：
 - 如果T不是redo-list中的事务，则什么也不做。
 - 如果T是redo-list中的事务，则为数据项X写入值V1。
- ◆③ 对每个未完成的事务，在日志中写入一个 $\langle T, ABORT \rangle$ 记录并刷新日志。



■ 恢复管理器对故障发生在不同时刻的不同处理

<T1 START> <T1,A,22> <T1,B,28.5> (a)	<T1 START> <T1,A,22> <T1,B,28.5> <T1,COMMIT> 若<T1 COMMIT> 在磁盘，同(C) (b) 若<T1 COMMIT>	<T1 START> <T1,A,22> <T1,B,28.5> <T1,COMMIT> 写A 到磁盘 (c)	<T1 START> <T1,A,22> <T1,B,28.5> <T1,COMMIT> 写A 到磁盘 写B 到磁盘 (d)
--	--	---	---

<T1 ABORT>

不在磁盘，同(a)

REDO

再次REDO



■ 后像前写：后像（AI）在事务提交前完全写入数据库

- ◆ 在日志中记录所有的数据库修改，一个事务的所有写操作在事务提交前已写入磁盘
- ◆ 在恢复时忽略已提交的事务；撤销未完成事务的影响。

■ 执行规则：

- ◆ 日志先写：如果事务T改变了数据项X，则记录变化的日志记录必须在数据项的新值写到磁盘前写入稳定的存储器；
- ◆ 如果事务提交，则<T COMMIT>日志记录必须在事务改变的所有数据项的新值写入到磁盘后再写入稳定的存储器，但应尽快。

事务T的执行步骤：

- 在T开始执行前，向日志中写入记录<T, START>
- T的一次write (X) 操作导致向日志中写入一条新记录
- 当T全部操作结束，被改变的所有数据项已写入磁盘后向日志中写入记录<T, COMMIT>。



■ 事务恢复过程

◆ Undo (T_i) : 将事务 T_i 更新的所有数据项的值设为旧值。

■ 简化日志记录内容：省去新值字段

◆ 日志记录 $\langle T, X, V1 \rangle$ 表示：事务 T 对数据项 X 执行写操作，写前的旧值为 $V1$ 。

步骤	动作	日志
1		$\langle T1 \text{ START} \rangle$
2	READ(A)	
3	$A := A + A * 0.1$	
4	Write (A)	$\langle T1, A, 20 \rangle$
5	READ(B)	
6	$B := B - B * 0.05$	
7	Write(B)	$\langle T1, B, 30 \rangle$
8	写A到磁盘	
9	写B到磁盘	
10		$\langle T1, \text{COMMIT} \rangle$



■ 恢复管理器执行步骤

- ◆ 首先对日志文件从后向前进行扫描，将有<T, COMMIT>记录的事务放入redo-list队列。
- ◆ 然后对日志文件从后向前进行扫描
 - 对遇到的每一个<T, X, V1>记录，若事务T在redo-list队列中，则恢复管理器什么都不做
 - 对遇到的每一个<T, X, V1>记录，若事务T不在redo-list队列中，则恢复管理器将数据项X在数据库中的值改为旧值V1。



恢复实现

日志

后像前写



■ 恢复管理器对故障发生在不同时刻的不同处理

<T1,START> <T1,A,20> <T1,B,30> 写A到磁盘 写B到磁盘 <T1,COMMIT> (a)	<T1,START> <T1,A,20> <T1,B,30> 写A到磁盘 写B到磁盘 (b)	<T1,START> <T1,A,20> <T1,B,30> 写A到磁盘 (c)	<T1,START> <T1,A,20> <T1,B,30> (d)
---	--	---	--

若<T1 COMMIT>在
磁盘，NO ACTION

UNDO

UNDO

UNDO

若<T1 COMMIT>不
在磁盘，UNDO

补<T1 ABORT>



■ 后像前后写：后像（AI）在事务提交前后写入数据库

- ◆ 被修改数据项写入磁盘的时间可以在日志记录 $\langle T, COMMIT \rangle$ 之前进行，也可以放在之后进行
- ◆ 写入的更新日志记录由4个部分组成。日志记录 $\langle T, X, V1, V2 \rangle$ 表示：事务 T 对数据项 X 执行写操作，写前的旧值为 $V1$ ，写后的新值为 $V2$

■ 执行规则

- ◆ 日志先写——被更新数据项写入磁盘前，更新记录 $\langle T, X, V1, V2 \rangle$ 必须已写到稳定存储器上。

■ 恢复过程

- ◆ Undo (T_i)：将未提交事务 T_i 更新的所有数据项的值设为旧值。
- ◆ Redo (T_i)：将已提交事务 T_i 更新的所有数据项的值设为新值。

步骤	动作	日志
1		$\langle T1 \text{ START} \rangle$
2	READ(A)	
3	$A := A + A * 0.1$	
4	Write (A)	$\langle T1, A, 20, 22 \rangle$
5	READ(B)	
6	$B := B - B * 0.05$	
7	Write(B)	$\langle T1, B, 30, 28.5 \rangle$
8	写A到磁盘	
9		$\langle T1, COMMIT \rangle$
10	写B到磁盘	



■ 恢复管理器执行步骤

- ◆ 首先对日志文件从后向前进行扫描，将有<T, COMMIT>记录和没有<T, COMMIT>记录的事务分别放入两个队列：redo-list队列，undo-list队列
- ◆ 从前向后再次扫描日志记录，重新执行redo-list队列中的事务。
- ◆ 从后向前再次扫描日志记录，撤销undo-list队列中的事务。



恢复实现

日志

后像前后写



■ 恢复管理器对故障发生在不同时刻的不同处理

<T1,START> <T1,A,20,22> <T1,B,30,28.5> (a)	<T1,START> <T1,A,20,22> <T1,B,30,28.5> 写A到磁盘 (b)	<T1,START> <T1,A,20,22> <T1,B,30,28.5> 写A到磁盘 <T1,COMMIT> (c)	
--	--	--	--

UNDO

UNDO

UNDO

或REDO



■ 等幂操作

- ◆ 多次执行与执行一次的效果是完全相同的。
- ◆ 无论回滚（Undo）操作还是重做（Redo）操作，恢复的步骤都是等幂的。
- ◆ 当系统从上一次崩溃中恢复时再次发生崩溃，恢复机制仍可再次重复恢复过程，前一次恢复过程中是否已将数据项的值更新为旧值或新值是无关紧要的。



- 利用日志进行数据库恢复时，恢复管理器必须检查日志，确定哪些事务需要REDO，哪些事务需要UNDO。一般来说，需要搜索整个日志才能做出决定。
- 搜索整个日志的困难
 - ◆ 搜索整个日志将耗费大量的时间。
 - ◆ 很多需要REDO处理的事务其更新已经写入数据库，重新执行这些操作会浪费大量的时间，使恢复过程变得更长。
- 解决办法
 - ◆ 系统定期或不定期地建立检查点（CheckPoint），保存数据库的状态。



恢复实现

日志

检查点



■ 创建检查点方法

- ◆ 提交一致性检查点
- ◆ 高速缓存一致性检查点
- ◆ 模糊一致性检查点



■ 提交一致性检查点

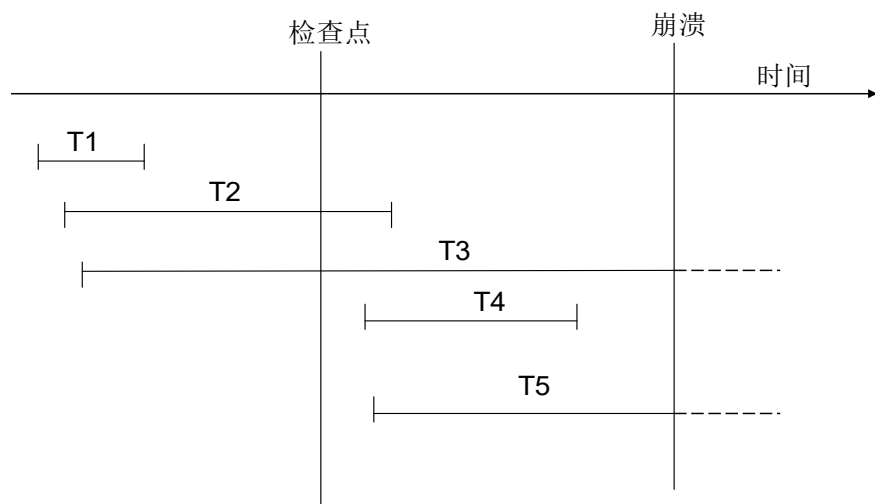
- ◆ 新的事务不能开始直到检查点完成。
- ◆ 现有的事务继续执行直到提交或中止，并且相关日志都写入稳定存储器。
- ◆ 将当前日志缓冲区中的日志记录写回稳定存储器中的日志文件。
- ◆ 将日志记录<checkpoint>写入稳定存储器。检查点操作完成。

```
< T1, START>  
< T1, A, 200>  
< T2, START>  
< T2, B, 10>  
< T1, C, 50>  
< T1, COMMIT>  
< T2, COMMIT>  
< CHECKPOINT>  
< T3, START>  
< T3, A, 5>
```



■ 高速缓存一致性检查点

- ◆ 新的事务不能开始直到检查点完成。
- ◆ 已存在的事务不允许执行新的更新操作，如写缓冲块或写日志记录。
- ◆ 将当前日志缓冲区中的日志记录写回稳定存储器中的日志文件。
- ◆ 将当前数据缓冲区中的所有数据记录写入磁盘。
- ◆ 将日志记录 $\langle \text{checkpoint}, L \rangle$ 写入稳定存储器，其中L是所有活动事务的列表。





■ 用户等待问题

- ◆ 高速缓存一致性检查点只需将所有当前数据缓冲区中的数据记录写入磁盘，而无需等待事务的提交，这相对于提交一致性检查点而言是一个提高。
- ◆ 但在这个过程中，所有事务不能执行动作，对于用户而言也就是需要等待。

■ 模糊一致性检查点

- ◆ 允许事务在<checkpoint>记录写入日志后但在修改过的缓存块写到磁盘前做更新。
- ◆ 系统将最后一个完善检查点记录在日志中的位置存在磁盘的固定位置last-checkpoint
- ◆ 系统在写入< CHECKPOINT>记录时不更新改信息，而是在写< CHECKPOINT>记录前，创建所有修改过的缓冲块的列表。
- ◆ 只有所有该列表中的缓冲块都输出到磁盘上以后，last-checkpoint信息才会更新。



■ DBA定期地将整个数据库复制到某种存储介质（如磁带、磁盘、光盘等）上保存起来的过程。

◆ 备用的数据文本称为后备副本或后援副本。

◆ 数据转储是数据库恢复中采用的基本技术。

■ 转储方法分类

◆ 静态转储与动态转储

◆ 完全转储与增量转储



恢复实现

数据转储

静态转储



- 在系统中无运行事务时进行转储
- 转储开始时数据库处于一致性状态
- 转储期间不允许对数据库的任何存取、修改活动
- 优点：实现简单
- 缺点：降低了数据库的可用性
 - ◆ 转储必须等用户事务结束
 - ◆ 新的事务必须等转储结束



恢复实现

数据转储

动态转储



- 转储操作与用户事务并发进行
- 转储期间允许对数据库进行存取或修改
- 优点
 - ◆ 不用等待正在运行的用户事务结束
 - ◆ 不会影响新事务的运行
- 缺点
 - ◆ 不能保证副本中的数据正确有效
- 利用动态转储得到的副本进行故障恢复
 - ◆ 需要把动态转储期间各事务对数据库的修改活动登记下来，建立日志文件
 - ◆ 后备副本加上日志文件才能把数据库恢复到某一时刻的正确状态



- **完全转储：每次转储全部数据库**
- **增量转储：只转储上次转储后更新过的数据**
- **完全转储与增量转储比较**
 - ◆ 从恢复角度看，使用完全转储得到的后备副本进行恢复往往更方便
 - ◆ 但如果数据库很大，事务处理又十分频繁，则增量转储方式更实用更有效



本章内容



1

事务概念

2

数据库恢复概述

3

数据库恢复实现技术



4

故障的种类及恢复策略

5

*RAID



■ 事务故障

- ◆ 逻辑错误：事务由于内部条件（如非法输入、溢出等）无法继续正常执行。
- ◆ 系统错误：系统进入一种不良状态（如死锁），事务无法继续正常执行。
- ◆ 事务故障使得事务无法达到预期的终点，数据库可能处于不一致的状态。恢复机制强行回滚该事务，撤销该事务对数据库做的任何修改。

■ 系统故障

- ◆ 包括硬件故障、数据库软件或操作系统的漏洞造成的系统停止运转。它导致系统易失性存储器中的内容丢失，事务处理停止，但非易失性存储器中的内容不会受到破坏。

■ 介质故障

- ◆ 在数据传送操作过程中由于磁头损坏或故障造成磁盘块上的内容丢失。
- ◆ 需使用其他非易失性存储器上的数据库后备副本进行故障的恢复。



■ 事务故障的恢复：利用日志文件撤销此事务对数据库已经进行做过的修改。

- ◆ 后像后写：发生故障时数据库中的数据并没有发生变化，所有数据项的修改只是在日志文件中有记录。恢复管理器忽略这些未完成的事务。
- ◆ 后像前写：发生故障时，系统可能已将部分或全部数据项的修改写入磁盘，使用日志文件撤销（UNDO）此事务对数据库的修改。
- ◆ 后像前后写：发生故障时系统仍可能已将部分数据项的修改写入磁盘。使用日志文件撤销（UNDO）此事务对数据库的修改。



■ 系统故障的恢复：利用日志文件回滚（UNDO）未完成事务，重做（REDO）已提交事务。

- ◆ 当系统崩溃重新启动时，它构造两个队列：undo-list存放需要撤销的事务标识符，redo-list存放需要重做得事务标识符。
- ◆ 这两个队列刚开始时都是空的。
- ◆ 队列构造步骤如下：
 - 系统反向扫描日志，直到发现第一个<checkpoint>记录。
 - 对每一个<Ti, COMMIT>记录，将Ti加入redo-list。
 - 对每一个<Ti, START>记录，如果Ti不属于redo-list，则将Ti加入undo-list。



■ 系统故障的恢复：后像后写

- ◆ 对undo-list中的事务，在日志中写入一个 $\langle Ti, ABORT \rangle$ 记录并刷新日志。
- ◆ 对redo-list中的事务执行REDO操作：从前面发现的 $\langle checkpoint \rangle$ 记录开始，正向扫描日志文件，对遇到的每一个 $\langle Ti, X, V1 \rangle$ 记录，将数据库中的X数据项更新为新值V1。

■ 系统故障的恢复：后像前写

- ◆ 对undo-list中的某一事务，执行UNDO操作：
- ◆ 再次反向扫描日志文件，对遇到的每一个 $\langle Ti, X, V1 \rangle$ 记录，将数据库中的X数据项更新为旧值V1，扫描到 $\langle Ti, START \rangle$ 记录时，扫描停止。
- ◆ 在日志中写入一个 $\langle Ti, ABORT \rangle$ 记录并刷新日志。
- ◆ 重复上两个步骤，直到处理完撤销队列中的每一个事务。



■ 系统故障的恢复：后像前后写

- ◆ 系统重新反向扫描日志文件，对undo-list中的每一事务执行UNDO操作，即对遇到的每一个 $\langle T_i, X, V1, V2 \rangle$ 记录，将数据库中的X数据项更新为旧值V1。
- ◆ 在日志中写入一个 $\langle T_i, ABORT \rangle$ 记录并刷新日志。当undo-list中所有事务 T_i 所对应的 $\langle T_i, START \rangle$ 记录都找到时，扫描结束。
- ◆ 系统找出日志中最后一条 $\langle checkpoint \rangle$ 记录。
- ◆ 系统由最后一条 $\langle checkpoint \rangle$ 记录开始，正向扫描日志文件，对redo-list中的事务 T_i 的每一个日志记录执行redo操作。即对遇到的每一个 $\langle T_i, X, V1, V2 \rangle$ 记录，将数据库中的X数据项更新为新值V2。



■ 装入最近的完全转储后备副本。

- ◆ 若数据库副本是动态转储的，还需要同时装入转储开始时刻的日志文件副本，利用恢复系统故障的方法将数据库恢复到某个一致性状态。

■ 如果有后续的增量转储，按照从前往后的顺序，根据增量转储来修改数据库。

■ 装入转储结束后的日志文件副本，重做已完成的事务。

。

- ◆ 首先反向扫描日志文件，找出故障发生时已经提交的事务，将事务标识符写入redo-list。
- ◆ 然后正向扫描日志文件，对redo-list中的所有事务进行redo操作。



本章内容



1

事务概念

2

数据库恢复概述

3

数据库恢复实现技术

4

故障的种类及恢复策略



5

*RAID



■ 冗余磁盘阵列 (Redundent Array of Independent Disks, RAID)

- ◆ RAID将一组磁盘驱动器用某种逻辑方式联系起来，作为逻辑上的一个磁盘驱动器来使用。
- ◆ 一般情况下，组成的逻辑磁盘驱动器的容量要小于各个磁盘驱动器容量的总和。
- ◆ RAID通过冗余技术，提供一个高级别的数据保护。
- ◆ RAID的具体实现可以靠硬件也可以靠软件，Windows NT操作系统就提供软件RAID功能。

■ 优点

- ◆ 成本低，功耗小，传输速率高
- ◆ 可以提供容错功能
- ◆ 具备数据校验 (Parity) 功能
- ◆ RAID比起传统的大直径磁盘驱动器来，在同样的容量下，价格要低许多



■ RAID的分级

- ◆ RAID 0 (Stripe)：无冗余无校验的磁盘阵列，并行读/写于多个磁盘上。
 - RAID 0只是单纯地提高性能，并没有为数据的可靠性提供保证，
 - 而且其中的一个磁盘失效将影响到所有数据。
- ◆ RAID 1 (Mirror)：镜象磁盘阵列，成对的独立磁盘上产生互为备份的数据。
 - RAID 1可以提高读取性能，提供最高的数据安全保障。
 - 备份数据占了总存储空间的一半，磁盘空间利用率低，存储成本高。
- ◆ RAID 0+1：也被称为RAID 10标准
 - 实际是将RAID 0和RAID 1标准结合的产物，在连续地以位或字节为单位分割数据并且并行读/写多个磁盘的同时，为每块磁盘做磁盘镜像。
 - RAID 1+0至少使用4个硬盘。
 - RAID 0+1是存储性能和数据安全兼顾的方案。



■ RAID的分级

◆ RAID 2：纠错海明码磁盘阵列

- 使用加重平均纠错码（海明码）的编码技术提供错误检查及恢复。
- RAID 2对大数据量的输入/输出有很高的性能，但少量数据的输入/输出时性能不好。
- RAID 2技术实施更复杂，因此在商业环境中很少使用。

◆ RAID 3/ RAID 4：奇校验或偶校验的磁盘阵列

- RAID 3/ RAID 4使用简单的奇偶校验，并用单块磁盘存放奇偶校验信息。
- RAID 3/RAID 4对于大量的连续数据可提供很好的传输率，但对于随机数据来说，奇偶盘会成为写操作的瓶颈。

◆ RAID 5：无独立校验盘的奇偶校验磁盘阵列

- 不单独指定奇偶盘，而是在所有磁盘上交叉地存取数据及奇偶校验信息。
- RAID 5更适合于小数据块和随机读/写的的数据。

◆ RAID 6：与RAID 5相比，RAID 6增加了第2个独立的奇偶校验信息块

◆ RAID 7：优化的高速数据传送磁盘结构

- 这是一种新的RAID标准，其自身带有智能化实时操作系统和用于存储管理的软件工具，可完全独立于主机运行，不占用主机CPU资源。
- RAID7系统成本很高，可以看做是一种存储计算机。



■ 常用RAID级别比较

RAID级别	RAID0	RAID1	RAID3	RAID5
容错性	没有	有	有	有
冗余类型	没有	复制	奇偶校验	奇偶校验
读性能	高	低	高	高
随即写性能	高	低	最低	低
连续写性能	高	低	低	低
需要的磁盘数	一个或多个	2个或2*N个	三个或更多	三个或更多
可用容量	总的磁盘容量	磁盘容量的50%	$(n-1)/n$ 的磁盘容量，N为磁盘数	$(n-1)/n$ 的磁盘容量，N为磁盘数
典型应用	无故障的迅速读写，安全性要求不高，如图形工作站等	随机数据写入，要求的安全性高，如数据库服务器在存储	连续数据传输，要求的安全性高。如视频编辑	随机数据传输，要求安全性高，如金融领域数据库



■ RAID级别的选择

- ◆ 失效概率
- ◆ 重建性能问题
- ◆ 调整的灵活性
- ◆ 缓存的使用
- ◆ 使用成本



本章小结



- 事务的ACID特性
- 日志的内容、作用
- 事务的执行与恢复过程
- 检查点
- 数据转储
- 故障分类
- 不同故障的恢复策略
- RAID